

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №2
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
«Розробка графічного редактора об'єктів на C#»

Виконав:
студент групи ІП-94
Матюшенко Анастасія Юріївна
Номер в списку - 11

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета роботи: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм, запрограмувавши простий графічний редактор в об'єктно-орієнтованому стилі.

Завдання:

1. Створити у середовищі **JetBrains Rider** проєкт типу **WPF Application** (.NET 9) з ім'ям Lab2.
2. Скомпілювати проєкт і отримати виконуваний файл програми.
3. Перевірити роботу програми, налагодити обробку подій миші та малювання фігур.
4. Проаналізувати та прокоментувати результати й вихідний текст програми з точки зору принципів ООП (інкапсуляція, успадкування, поліморфізм) та вимог 11-го варіанта.
5. Оформити звіт.

Завдання згідно варіанту:

- **Зберігання фігур:** Статичний масив вказівників на 111 елементів ($11 + 100$).
- **Гумовий слід:** Пунктирна лінія чорного кольору ($11 \bmod 4 = 3$).
- **Прямокутник:**
 - Ввід: Від центру до одного з кутів ($11 \bmod 2 = 1$).
 - Відображення: Чорний контур із сірим заповненням ($11 \bmod 5 = 1$; $11 \bmod 6 = 5$).
- **Еліпс:**
 - Ввід: За двома протилежними кутами охоплюючого прямокутника ($11 \bmod 2 = 1$).
 - Відображення: Чорний контур із білим заповненням ($11 \bmod 5 = 1$).
- **Позначка поточного типу об'єкта:** Відображається в заголовку вікна ($11 \bmod 2 = 1$).

Вихідні тексти програми:

Shape.cs

```
using System;
using System.Windows.Shapes;
using System.Windows.Media;
using System.Windows.Controls;

namespace Lab2
{
    public abstract class Shape
    {
        protected double x1, y1, x2, y2;

        public void Set(double startX, double startY, double endX, double
endY)
        {
```

```

        x1 = startX;
        y1 = startY;
        x2 = endX;
        y2 = endY;
    }

    public abstract void Draw(Canvas canvas);

    public abstract void Update(System.Windows.UIElement element);
}

public class PointShape : Shape
{
    public override void Draw(Canvas canvas)
    {
        var ellipse = new Ellipse()
        {
            Width = 4,
            Height = 4,
            Fill = Brushes.Black
        };

        Canvas.SetLeft(ellipse, x1);
        Canvas.SetTop(ellipse, y1);

        canvas.Children.Add(ellipse);
    }

    public override void Update(System.Windows.UIElement element)
    {
    }
}

public class LineShape : Shape
{
    public override void Draw(Canvas canvas)
    {
        var line = new Line()
        {
            X1 = x1,
            Y1 = y1,
            X2 = x2,
            Y2 = y2,
            Stroke = Brushes.Black,
            StrokeThickness = 2
        };
        canvas.Children.Add(line);
    }

    public override void Update(System.Windows.UIElement element)
    {
        if (element is Line line)
        {
            line.X2 = x2;
            line.Y2 = y2;
        }
    }
}

public class EllipseShape : Shape
{
    public override void Draw(Canvas canvas)
    {
        var ellipse = new Ellipse()

```

```

        {
            Width = Math.Abs(x2 - x1),
            Height = Math.Abs(y2 - y1),
            Stroke = Brushes.Black,
            Fill = Brushes.White,
            StrokeThickness = 2
        };

        Canvas.SetLeft(ellipse, Math.Min(x1, x2));
        Canvas.SetTop(ellipse, Math.Min(y1, y2));

        canvas.Children.Add(ellipse);
    }

    public override void Update(System.Windows.UIElement element)
    {
        if (element is Ellipse ellipse)
        {
            ellipse.Width = Math.Abs(x2 - x1);
            ellipse.Height = Math.Abs(y2 - y1);
            Canvas.SetLeft(ellipse, Math.Min(x1, x2));
            Canvas.SetTop(ellipse, Math.Min(y1, y2));
        }
    }
}

public class RectShape : Shape
{
    public override void Draw(Canvas canvas)
    {
        double width = Math.Abs(x2 - x1) * 2;
        double height = Math.Abs(y2 - y1) * 2;

        var rect = new Rectangle()
        {
            Width = width,
            Height = height,
            Stroke = Brushes.Black,
            Fill = new SolidColorBrush(Color.FromRgb(192, 192, 192)),
            StrokeThickness = 2
        };

        Canvas.SetLeft(rect, x1 - Math.Abs(x2 - x1));
        Canvas.SetTop(rect, y1 - Math.Abs(y2 - y1));

        canvas.Children.Add(rect);
    }

    public override void Update(System.Windows.UIElement element)
    {
        if (element is Rectangle rect)
        {
            rect.Width = Math.Abs(x2 - x1) * 2;
            rect.Height = Math.Abs(y2 - y1) * 2;
            Canvas.SetLeft(rect, x1 - Math.Abs(x2 - x1));
            Canvas.SetTop(rect, y1 - Math.Abs(y2 - y1));
        }
    }
}
}

```

```

using System.Windows.Input;
using System.Windows.Controls;

namespace Lab2
{
    public abstract class Editor
    {
        public abstract void OnMouseDown(Canvas canvas, MouseButtonEventArgs
e);
        public abstract void OnMouseMove(Canvas canvas, MouseEventArgs e);
        public abstract void OnMouseUp(Canvas canvas, MouseButtonEventArgs
e);
    }

    public class ShapeEditor : Editor
    {
        private Shape[] pcshape = new Shape[111];
        private int shapeCount = 0;
        protected double x1, y1, x2, y2;
        protected bool isDrawing = false;
        private System.Windows.UIElement previewElement;

        private Shape tempShape;

        public string CurrentShapeType { get; set; } = "Line";

        public override void OnMouseDown(Canvas canvas, MouseButtonEventArgs
e)
        {
            var pos = e.GetPosition(canvas);
            x1 = pos.X;
            y1 = pos.Y;
            x2 = x1;
            y2 = y1;
            isDrawing = true;

            tempShape = CurrentShapeType switch
            {
                "Point" => new PointShape(),
                "Line" => new LineShape(),
                "Rect" => new RectShape(),
                "Ellipse" => new EllipseShape(),
                _ => new LineShape()
            };

            tempShape.Set(x1, y1, x2, y2);

            previewElement = CreateWpfElement(tempShape);

            if (previewElement is System.Windows.Shapes.Shape wpfShape)
            {
                wpfShape.StrokeDashArray = new
System.Windows.Media.DoubleCollection { 2, 2 };
            }

            if (previewElement != null)
            {
                canvas.Children.Add(previewElement);
            }
        }

        public override void OnMouseMove(Canvas canvas, MouseEventArgs e)
        {
            if (!isDrawing || previewElement == null || tempShape == null)

```

```

return;

        var pos = e.GetPosition(canvas);
        x2 = pos.X;
        y2 = pos.Y;

        tempShape.Set(x1, y1, x2, y2);
        tempShape.Update(previewElement);
    }

    public override void OnMouseUp(Canvas canvas, MouseButtonEventArgs e)
    {
        if (!isDrawing) return;

        var pos = e.GetPosition(canvas);
        x2 = pos.X;
        y2 = pos.Y;
        isDrawing = false;

        if (previewElement != null)
        {
            canvas.Children.Remove(previewElement);
            previewElement = null;
        }

        tempShape.Set(x1, y1, x2, y2);
        tempShape.Draw(canvas);

        if (shapeCount < 111)
        {
            pcsshape[shapeCount] = tempShape;
            shapeCount++;
        }

        tempShape = null;
    }

    private System.Windows.UIElement CreateWpfElement(Shape shape)
    {
        var tempCanvas = new Canvas();
        shape.Draw(tempCanvas);
        if (tempCanvas.Children.Count > 0)
        {
            var el = tempCanvas.Children[0];
            tempCanvas.Children.Clear();
            return el;
        }
        return null;
    }
}
}

```

MainWindow.xaml

```

<Window x:Class="Lab2.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        xmlns:local="clr-namespace:Lab2"
        mc:Ignorable="d"
        Title="Лабораторна робота №2 - Матюшенко Анастасія" Height="450"

```

```

Width="800">
    <Grid>
        <DockPanel>
            <Menu DockPanel.Dock="Топ">
                <MenuItem Header="Файл">
                    <MenuItem Header="Вихід" Click="Exit_Click"/>
                </MenuItem>
                <MenuItem Header="Об'єкти">
                    <MenuItem Header="Крпка" Click="Point_Click"/>
                    <MenuItem Header="Лінія" Click="Line_Click"/>
                    <MenuItem Header="Еліпс" Click="Ellipse_Click"/>
                    <MenuItem Header="Прямокутник" Click="Rectangle_Click"/>
                </MenuItem>
                <MenuItem Header="Довідка">
                    <MenuItem Header="Про програму" Click="About_Click"/>
                </MenuItem>
            </Menu>

            <Canvas x:Name="drawingCanvas" Background="White"
                MouseLeftButtonDown="drawingCanvas_MouseLeftButtonDown"
                MouseMove="drawingCanvas_MouseMove"
                MouseLeftButtonUp="drawingCanvas_MouseLeftButtonUp"/>
        </DockPanel>
    </Grid>
</Window>

```

MainWindow.xaml.cs

```

using System.Windows;
using System.Windows.Input;

namespace Lab2
{
    public partial class MainWindow : Window
    {
        private ShapeEditor shapeEditor = new ShapeEditor();

        public MainWindow()
        {
            InitializeComponent();
        }

        private void Exit_Click(object sender, RoutedEventArgs e)
        {
            Application.Current.Shutdown();
        }

        private void Point_Click(object sender, RoutedEventArgs e)
        {
            shapeEditor.CurrentShapeType = "Point";
            Title = "Лабораторна робота №2 - Режим: Крпка";
        }

        private void Line_Click(object sender, RoutedEventArgs e)
        {
            shapeEditor.CurrentShapeType = "Line";
            Title = "Лабораторна робота №2 - Режим: Лінія";
        }

        private void Ellipse_Click(object sender, RoutedEventArgs e)
        {
            shapeEditor.CurrentShapeType = "Ellipse";
            Title = "Лабораторна робота №2 - Режим: Еліпс";
        }
    }
}

```

```

    }

    private void Rectangle_Click(object sender, RoutedEventArgs e)
    {
        shapeEditor.CurrentShapeType = "Rect";
        Title = "Лабораторна робота №2 - Режим: Прямокутник";
    }

    private void About_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show("Лабораторна робота №2\nВиконала: Матюшенко  
Анастасія\nВаріант 11",
            "Довідка",
            MessageBoxButton.OK,
            MessageBoxImage.Information);
    }

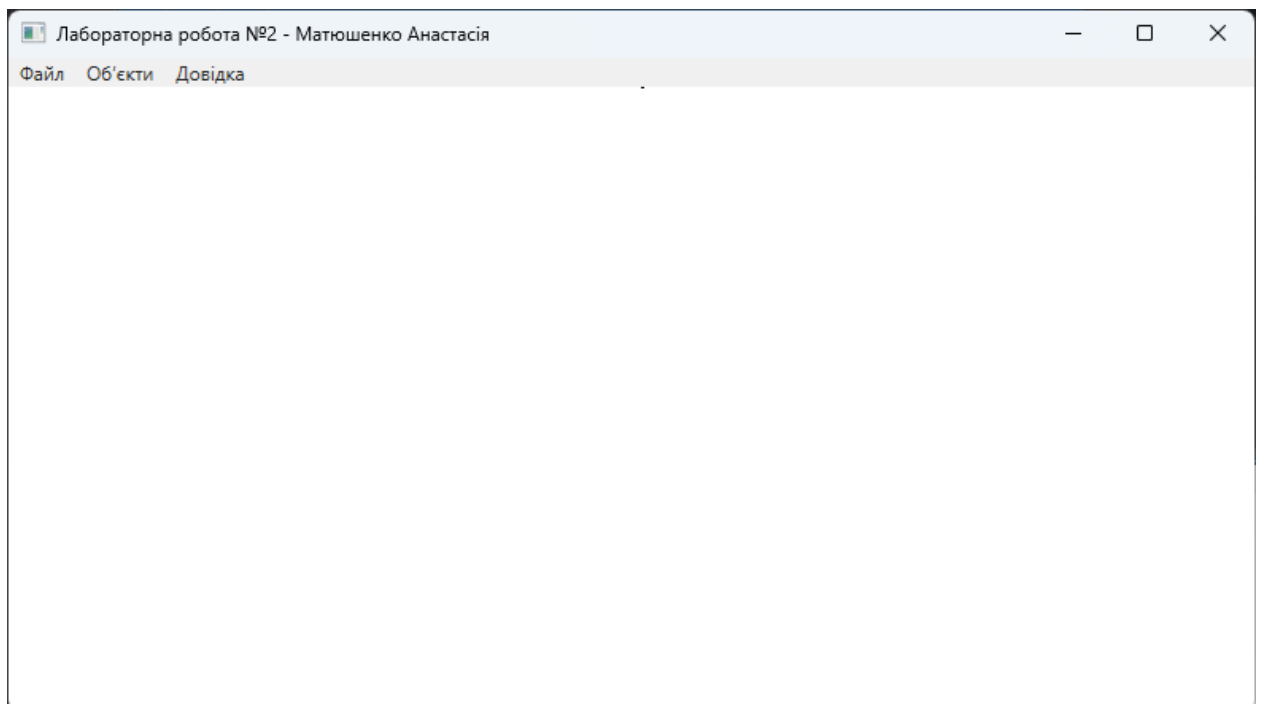
    private void drawingCanvas_MouseLeftButtonDown(object sender,
        MouseButtonEventArgs e)
    {
        shapeEditor.OnMouseDown(drawingCanvas, e);
    }

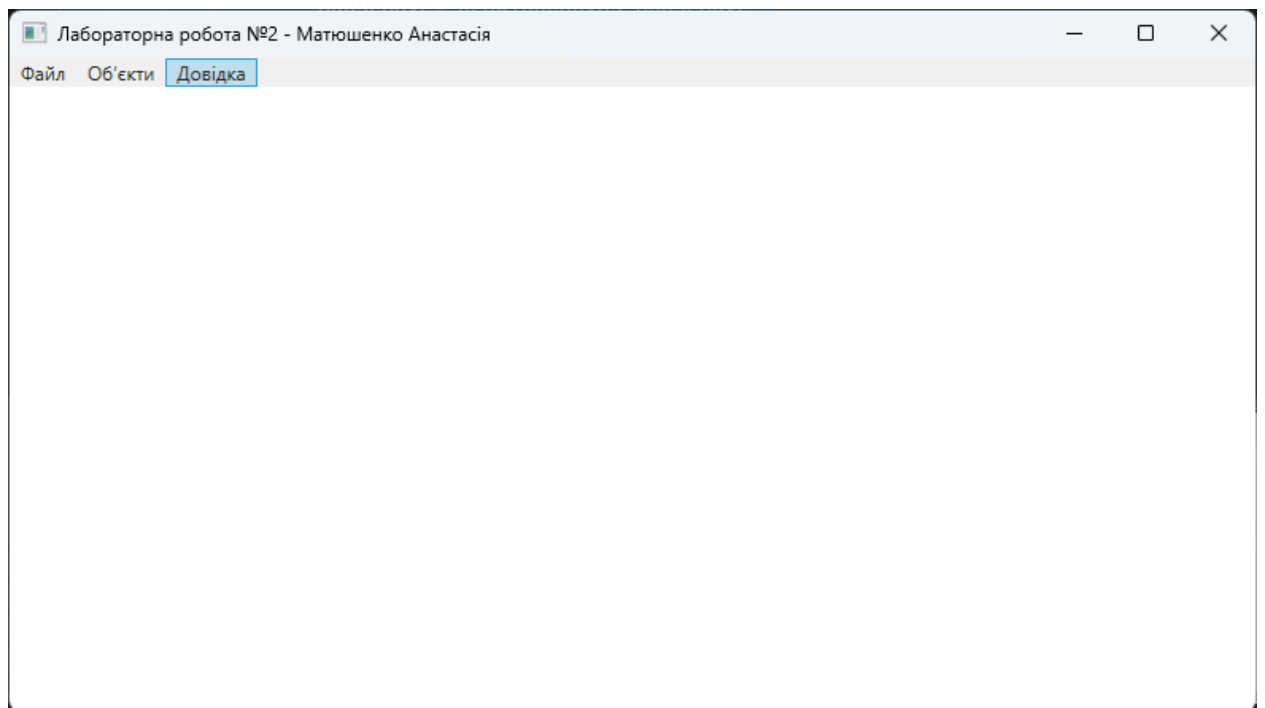
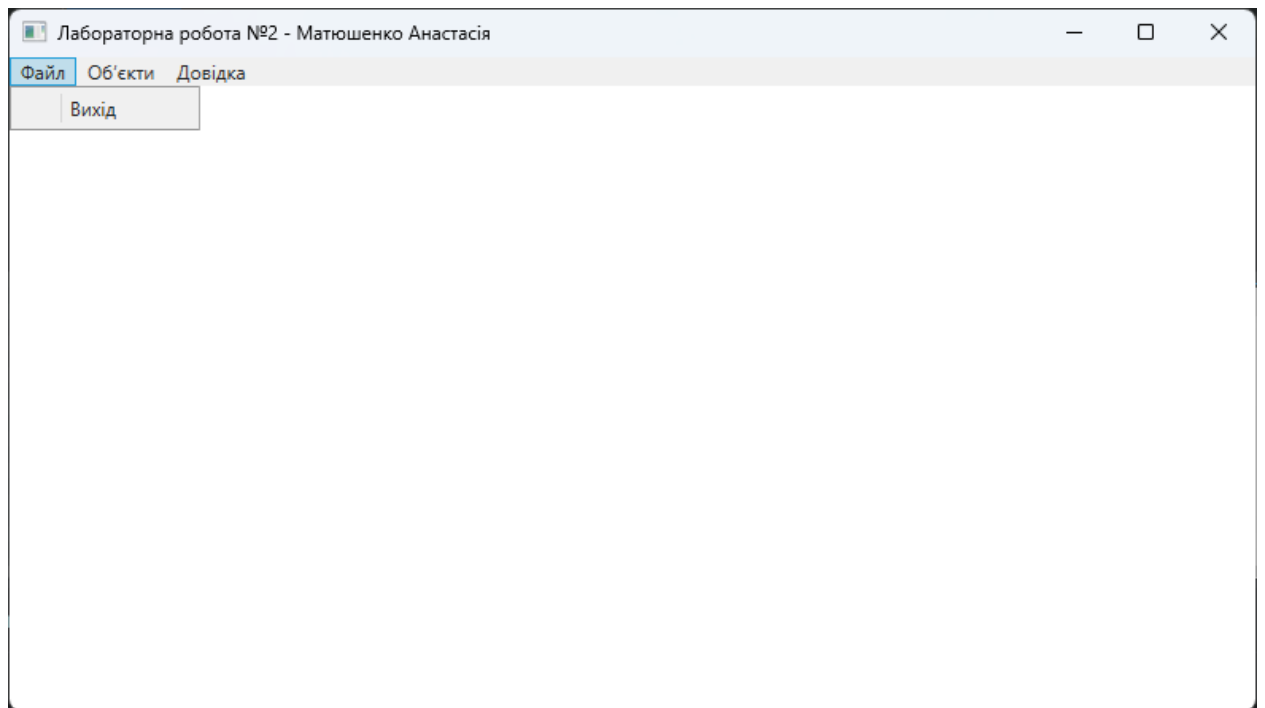
    private void drawingCanvas_MouseMove(object sender, MouseEventArgs e)
    {
        shapeEditor.OnMouseMove(drawingCanvas, e);
    }

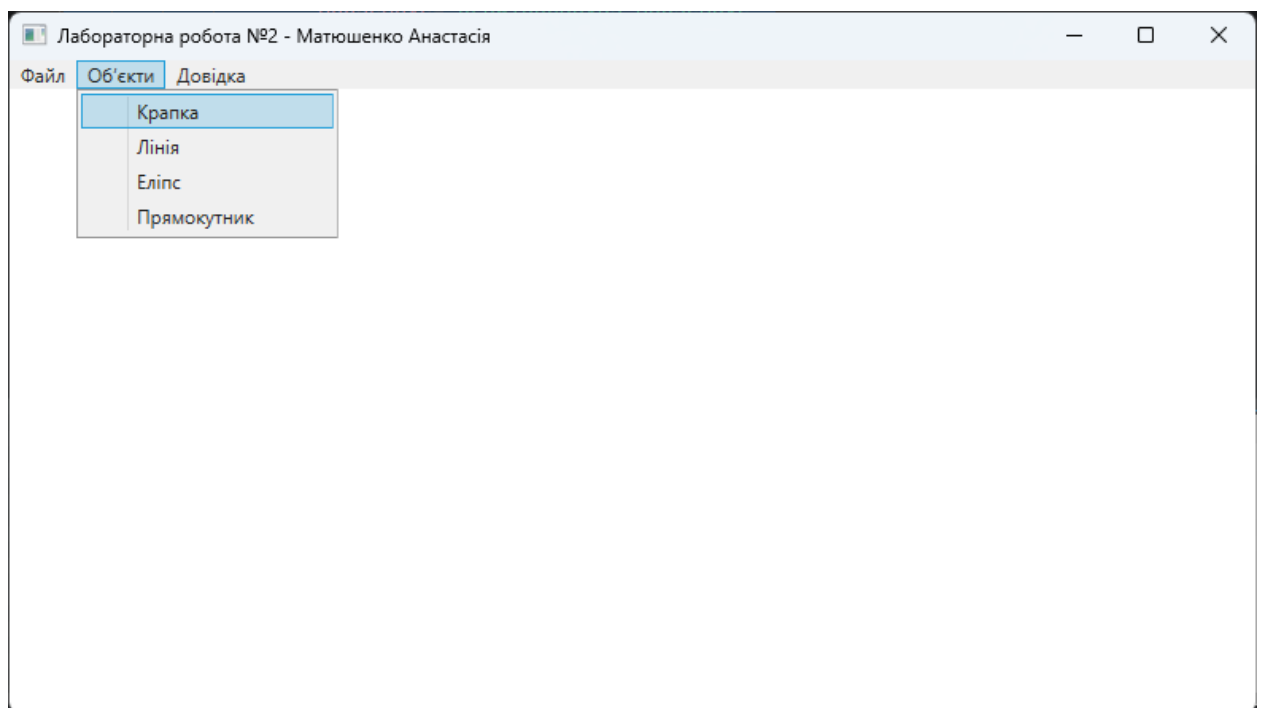
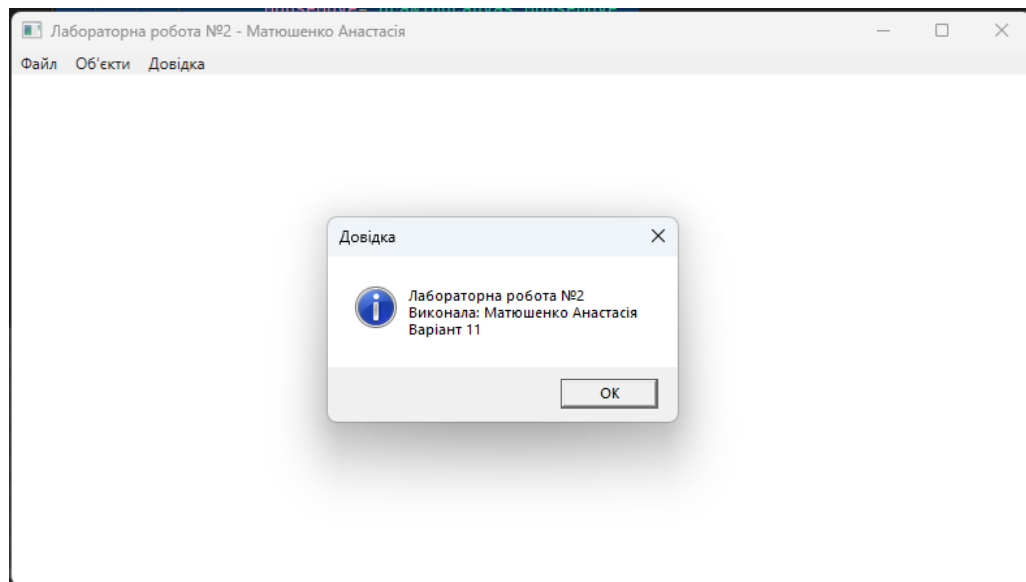
    private void drawingCanvas_MouseLeftButtonUp(object sender,
        MouseButtonEventArgs e)
    {
        shapeEditor.OnMouseUp(drawingCanvas, e);
    }
}

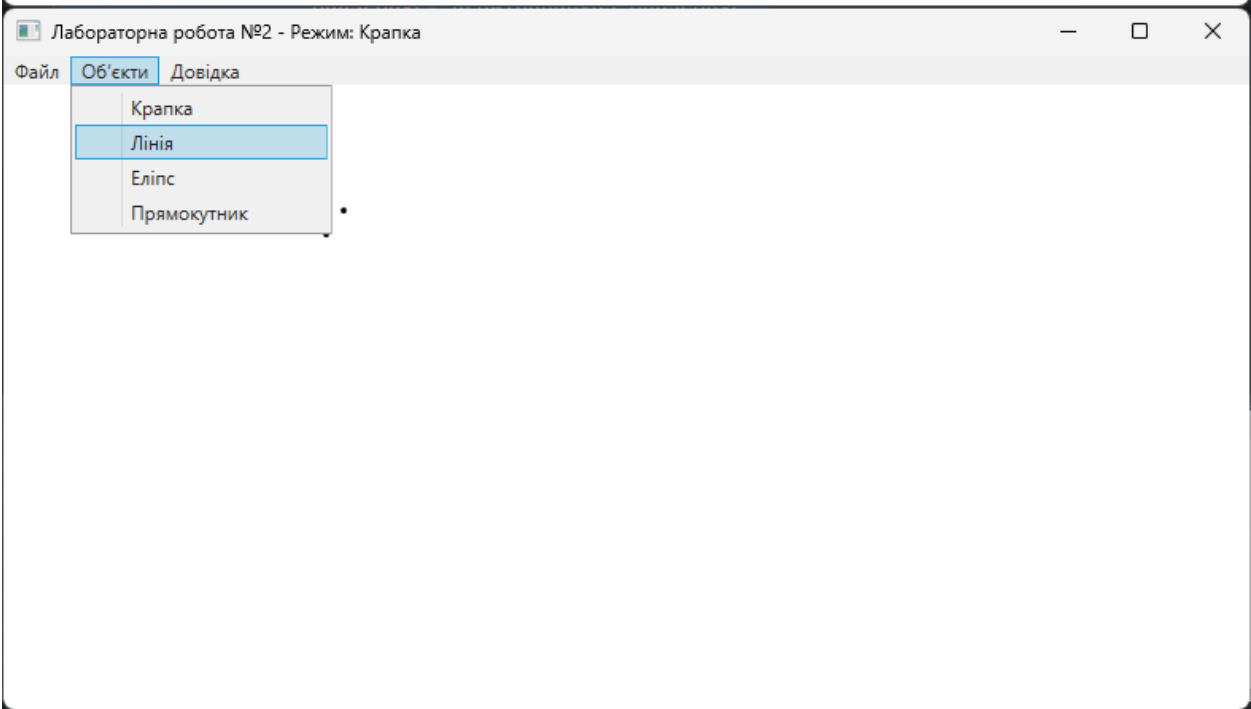
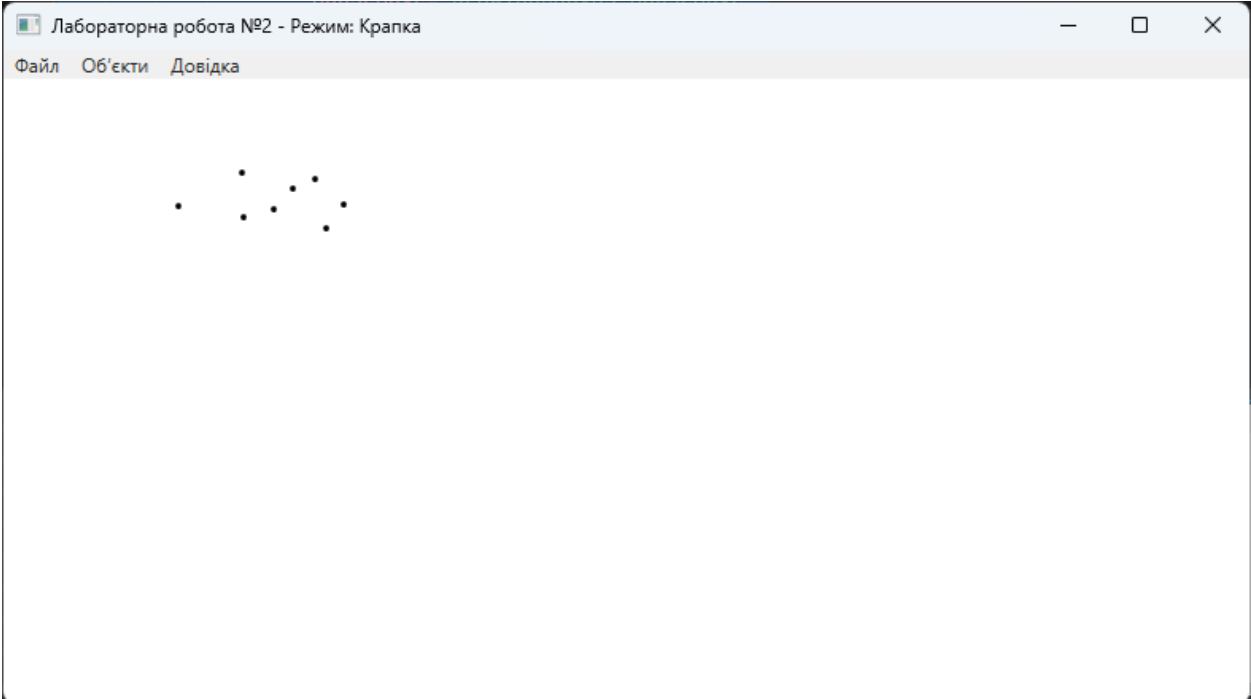
```

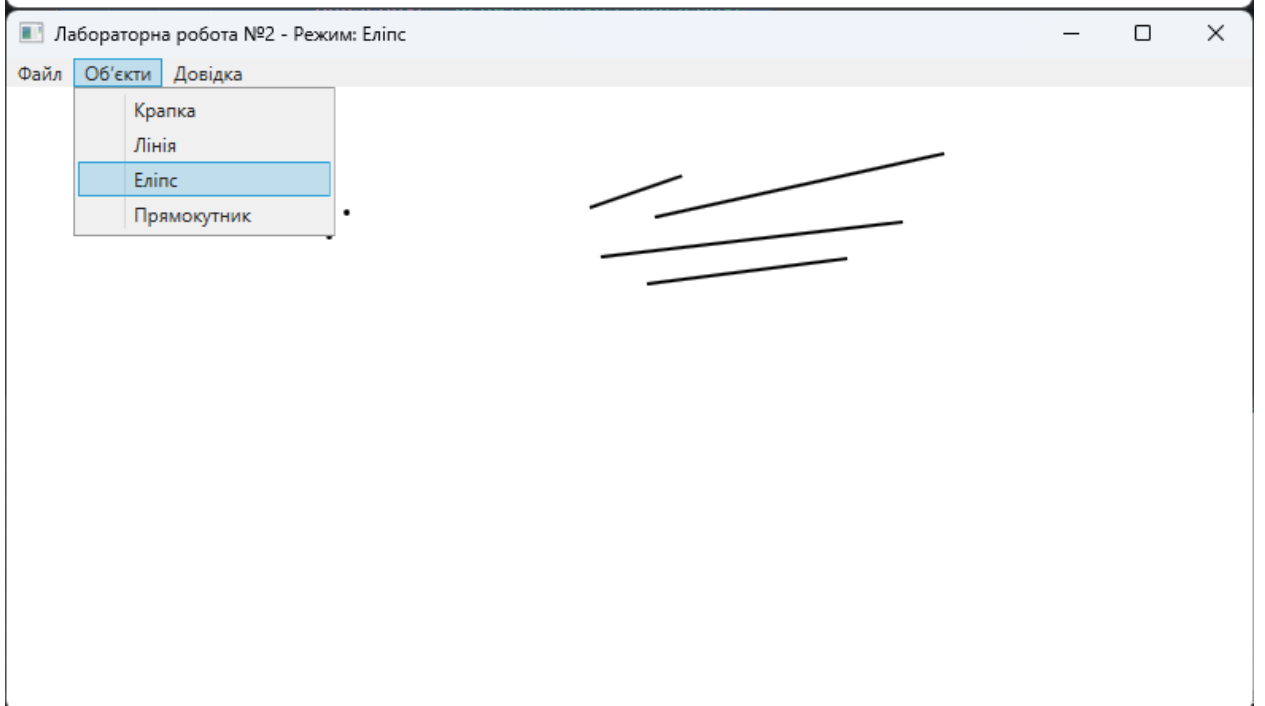
Скріншоти:

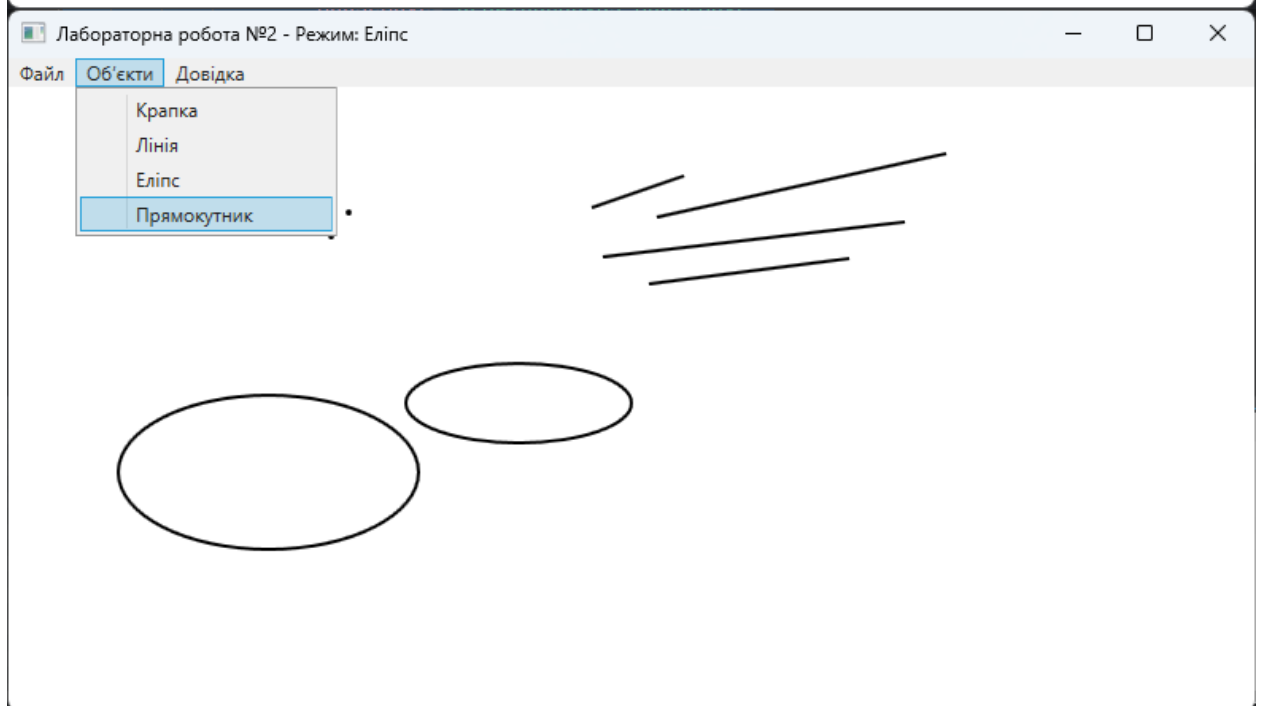
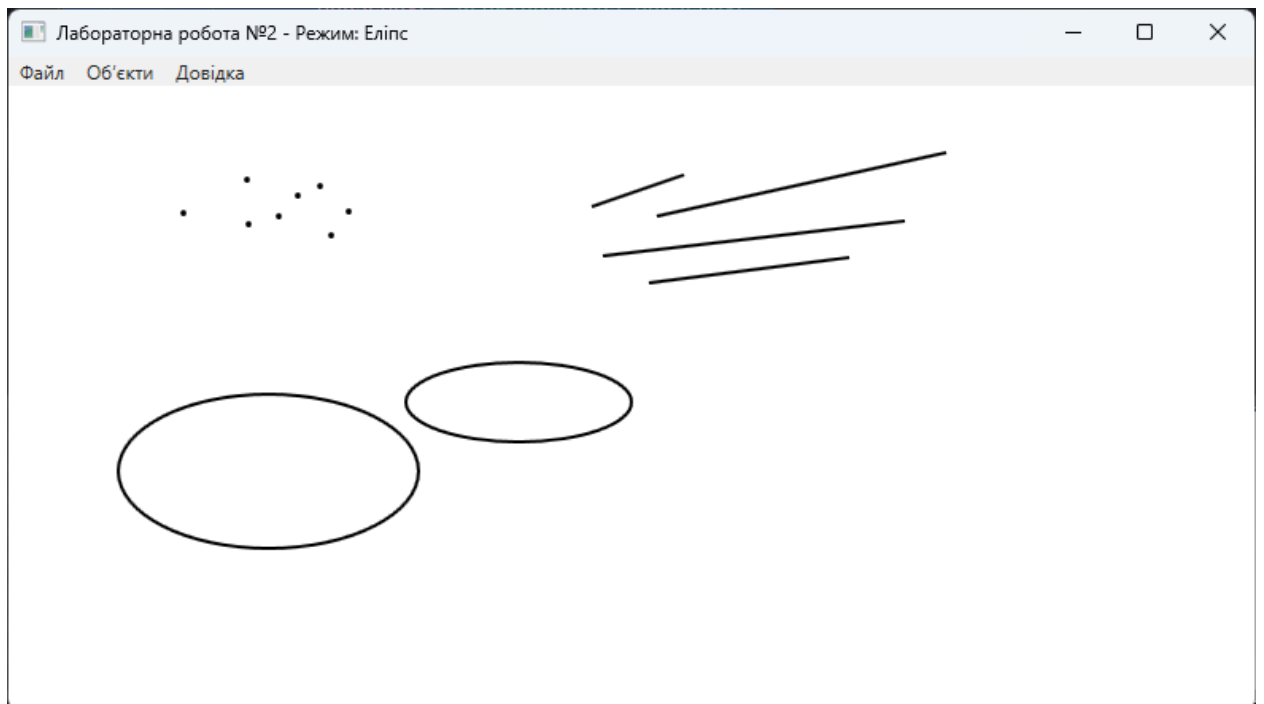


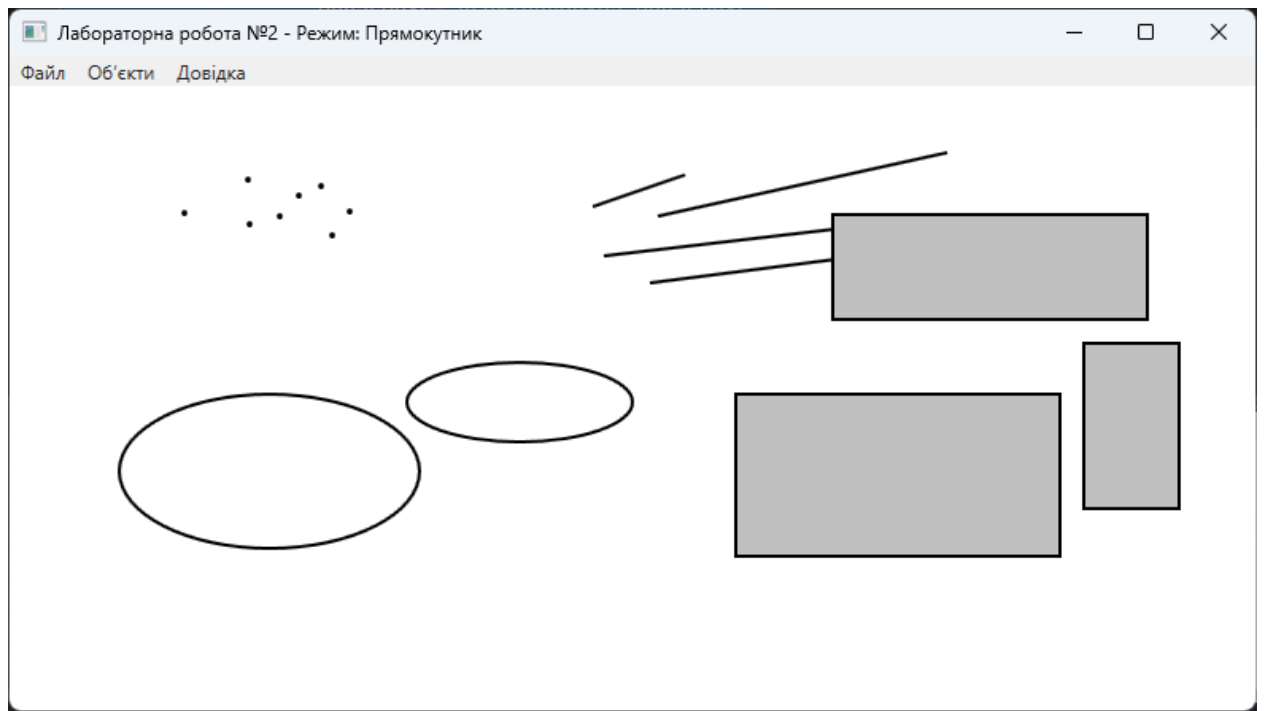




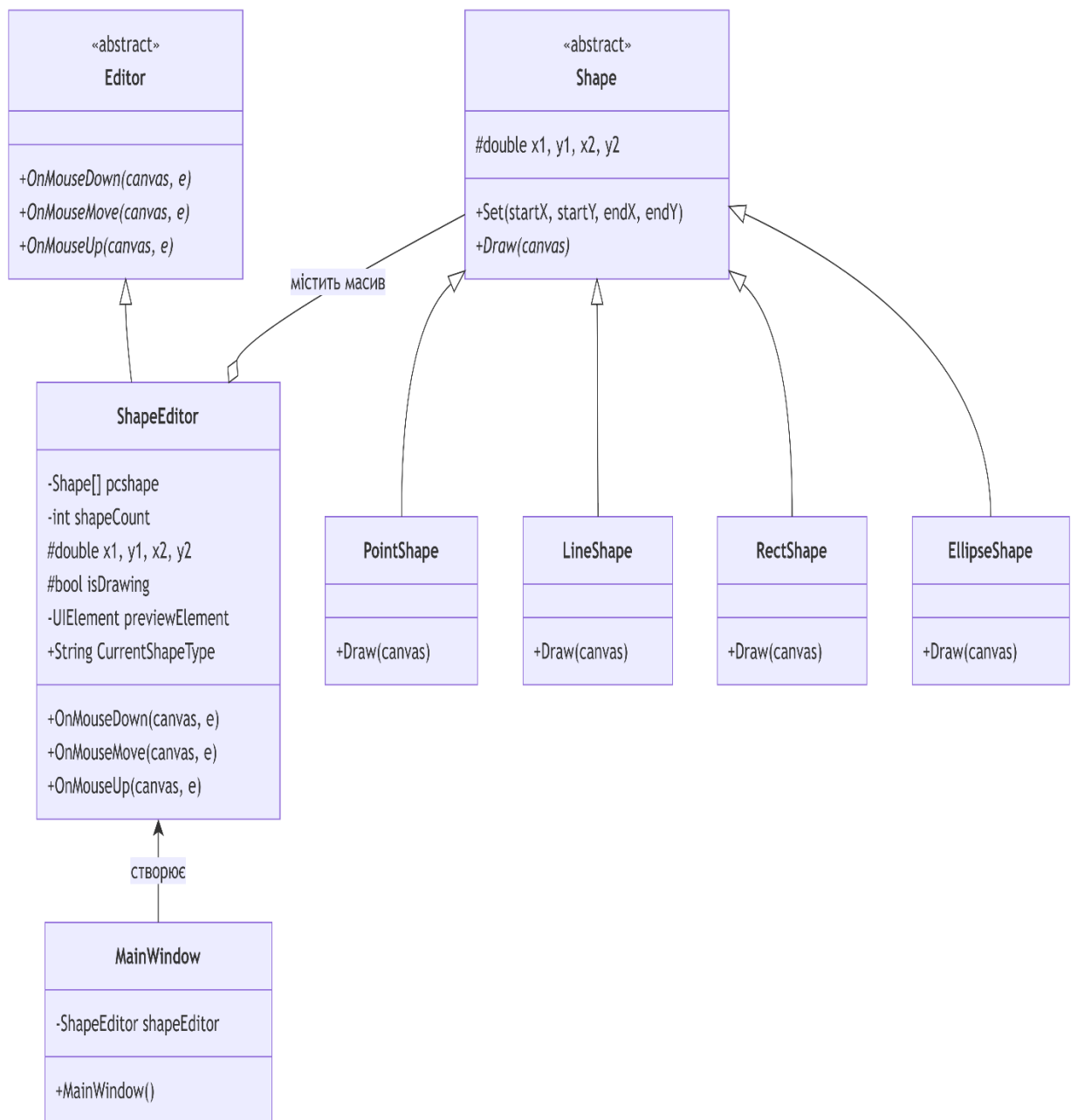








Діаграма залежностей файлів та модулів:



Висновки: у ході лабораторної роботи було отримано вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм. Також було запрограмовано простий графічний редактор в об'єктно-орієнтованому стилі.